

```
from tensorflow.keras.datasets import cifar10
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
(cifar_x, _), _ = cifar10.load_data()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 2218s 13us/step

```
input_size = 32
filter_size = 3

stride = 1
padding = 0

output_size = ((input_size - filter_size + (2 * padding)) // stride) + 1

print(output_size)

output_size = ((input_size - filter_size + (2 * padding)) // stride) + 1

output_size
```

30

```
image = cifar_x[0]
print(f"Size of image: {image.shape}")

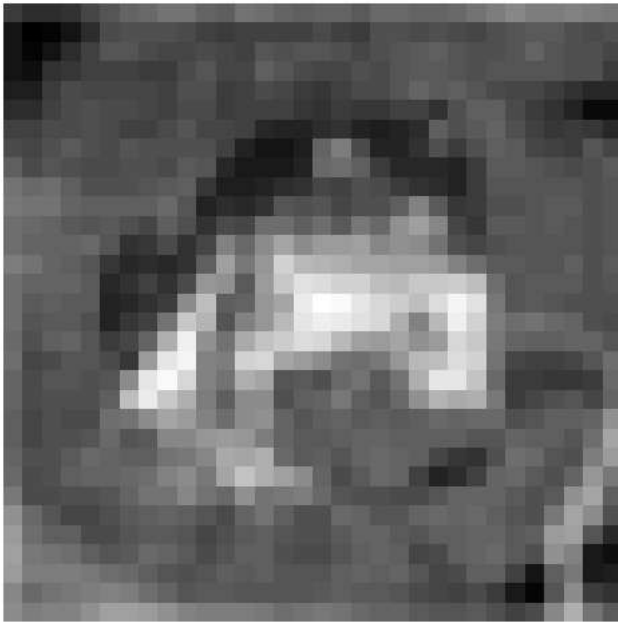
plt.figure(figsize=(10, 6))
plt.imshow(image)
plt.axis('off')
plt.show()
```

Size of image: (32, 32, 3)



```
grey = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
grey.shape

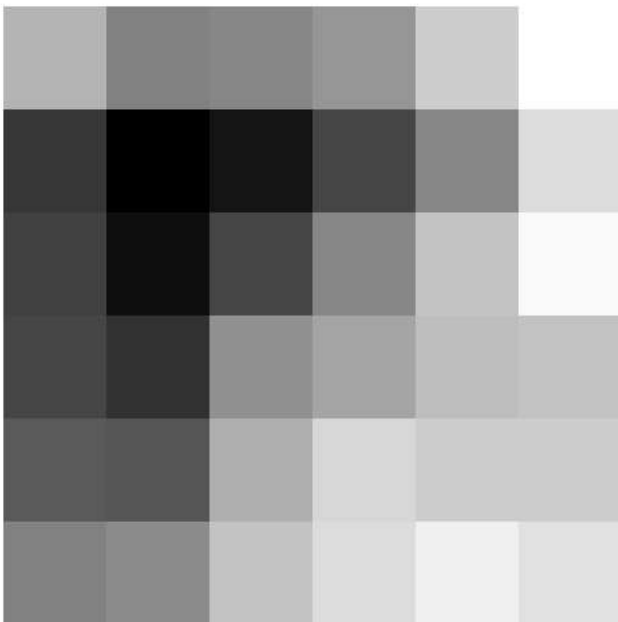
plt.figure(figsize=(10, 6))
plt.imshow(grey, cmap='gray')
plt.axis('off')
plt.show()
```



```
extract_patch = grey[0:6, 0:6]

extract_patch

plt.figure(figsize=(10, 6))
plt.imshow(extract_patch, cmap='gray')
plt.axis('off')
plt.show()
```



```
from numpy import dtype
kernal = np.array([
    [1, 1, 1],
    [1, 1, 1],
    [1, 1, 1]
], dtype=np.float32)

kernal = kernal / 9
print(kernal)

[[0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]]
```

```
def convolution(image, kernel, padding=0, stride=1):

    image = np.pad(image, pad_width=padding, mode='constant', constant_values=0)

    img_h, img_w = image.shape[0], image.shape[1]
    kernal_h, kernal_w = kernel.shape[0], kernel.shape[1]

    output_h, output_w = (img_h - kernal_h) // stride + 1, (img_w - kernal_w) // stride + 1

    output = np.zeros((output_h, output_w))

    for r in range(output_h):
        for c in range(output_w):

            y = r * stride
            x = c * stride

            region = image[y: y + kernal_h, x: x + kernal_w]

            output[r, c] = np.sum(region * kernel)

    return output
```

```
# ouput without padding

output = convolution(image=extract_patch, kernel=kernel, padding=0, stride=1)
output.shape

output1 = convolution(image=extract_patch, kernel=kernel, padding=1, stride=1)
output1.shape

for out in [output, output1]:
    plt.figure(figsize=(10, 6))
    plt.imshow(out, cmap='gray')
    plt.axis('off')
    plt.show()
```

